

ID1200: Operating Systems

Assignment 2

Fall 2025

Author(s): Andreas S. Pedersen
Vasudev Karunakaran
Padmini Thirumalai Ananthan Pillai
Date: October 14, 2025

Contents

1	Question 1 - Scheduling Algorithms	1
2	Question 2 - Scheduler Criteria	3
3	Question 3 - Thread creation, termination, and synchronization	5
4	Question 4 - Multi-threaded Histogram	9
5	Question 5 - Lock-free Synchronization	13
6	Question 6 - Resource-Allocation Graph	19
7	Question 7 - Banker's Algorithm	21
8	Question 8 - Minimum Resources	25

Question 1 - Scheduling Algorithms

Consider the following set of processes, with the length of the CPU burst given in milliseconds. A smaller priority number implies a higher priority.

Process	Arrival Time	Burst Time	Priority
P1	0	10	4
P2	1	3	3
P3	2	5	2
P4	6	1	0

For each of the following scheduling algorithms FCFS, preemptive SRTF, non-preemptive priority scheduling, and Round-Robin (quantum = 4):

- Draw the Gantt chart to illustrate the execution of these schedules.
- Compute the **turnaround time** of each process for each scheduling algorithm.
- Compute the **waiting time** of each process for each scheduling algorithm.
- Determine which algorithm results in the **minimum average waiting time**.

Solution

a) Gantt Charts

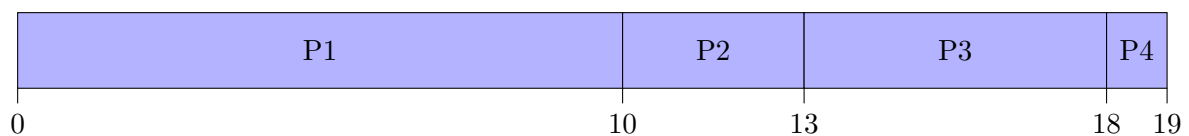


Figure 1: First Come First Served (FCFS)

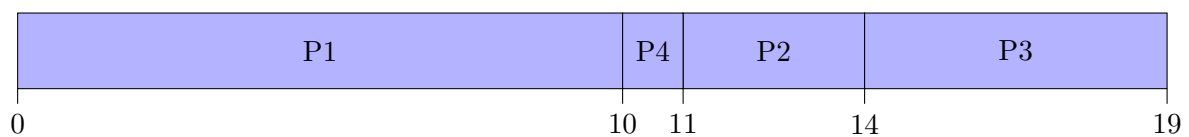


Figure 2: Shortest Job first (SJF)

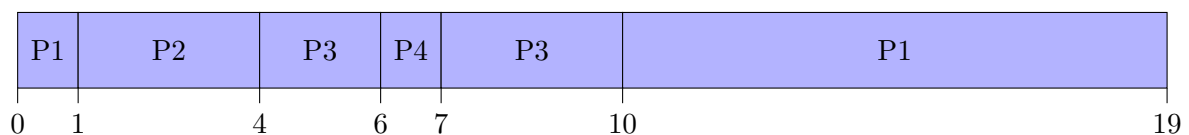


Figure 3: Preemptive Shortest Remaining Time First (SRTF)

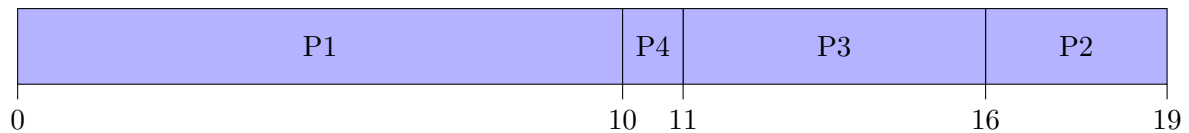


Figure 4: Non-preemptive Priority Scheduling

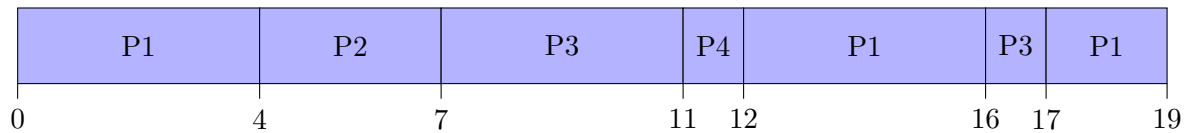


Figure 5: Round Robin (quantum = 4)

b) Turnaround Times

Algorithm	P1	P2	P3	P4
FCFS	10	3	5	1
SJF	10	3	5	1
Preemptive SRTF	19	3	6	1
Non-preemptive Priority	10	3	5	1
Round-Robin (Q=4)	19	3	10	1

c) Waiting Times

Algorithm	P1	P2	P3	P4
FCFS	0	9	11	12
SJF	0	13	12	4
Preemptive SRTF	0	0	2	0
Non-preemptive Priority	0	9	9	10
Round-Robin (Q=4)	0	3	5	5

d) Minimum Average Waiting Times

The average waiting times of the algorithms are:

- FCFS: $\frac{0+9+11+12}{4} = 8$
- SJF: $\frac{0+13+12+4}{4} = 7.25$
- Preemptive SRTF: $\frac{0+0+2+0}{4} = 0.5$
- Non-preemptive Priority: $\frac{0+9+9+10}{4} = 5.25$
- Round-Robin: $\frac{0+3+5+5}{4} = 3.25$

Thus, **Preemptive SRTF** has the minimum average waiting time.

Question 2 - Scheduler Criteria

Provide concrete examples to illustrate scenarios when the following scheduling criteria conflict:

- a) Average Turnaround Time vs Maximum Waiting Time
- b) Response Time vs Maximum Waiting Time
- c) System Job Throughput vs Response Time

Solution

a) Average Turnaround Time and Maximum Waiting Time

In this case, if we want to minimise average turnaround time and maximum waiting time, it can lead to conflicts. If process A is currently running, and the waiting time of some process in the ready queue has reached its maximum, process A will be interrupted and replaced by the new process. In one example, If we minimise the maximum waiting time to some minimum value k , and set the number of processes in the ready queue to be large, there will be a large number of new processes replacing previous processes. It will take a long time until a single process is fully completed, thus increasing the average turnaround time (this is a conflict).

In another example, if we want to minimise the average turnaround time, we cannot have too much context switching between processes. Processes will most likely run indefinitely until process finishes. If a particular process takes a very long time to run, the waiting time of the other processes will increase. The maximum waiting time must thus be increased to accommodate less context switching. Average turnaround time is reduced but maximum waiting time is increased.

b) Response Time and Maximum Waiting Time

In this case, if we want to minimise response time and maximum waiting time, it can lead to conflicts. One example that will be given here is similar to the one above. Assume we set the number of processes in the ready queue to be large, and have their burst times be similar but much larger than their maximum waiting time. We assume the maximum waiting time in this case is minimised. Once the first process is run, it won't take long until it is replaced by another process in the queue. This process of replacing continues to happen again and again. Given that the number of processes to run is large, it will take a long time until a single process is thoroughly completed for the first time, thus increasing response time.

Another example is if we wish to minimise response time. Imagine a scenario of time-consuming processes of similar length. To minimise response time, the processes should run without interruption until termination, as interruptions create overhead, delaying any process from being completed first. When a process runs without interruption, the waiting time of the remaining processes increases. If we try to minimise the maximum waiting time in this case, more interruptions will be caused, thus increasing response time. Minimising one side increases the other side. This is a conflict.

c) System Job Throughput and Response Time

In this case, if we want to minimise response time and maximise throughput, it can lead to conflicts. This is mainly caused by the overhead created when we do context switching.

An example would be: Consider a system where there are “quick” processes and “slow” processes running, e.g. retrieving an HTML file and rendering a large file respectively. Imagine the slow process is running in the background. If we minimise response time, when the call for the quick process is received, to reduce response time and have a response returned as soon as possible, the slow process will be switched to the quick process and switched back once completed. This indeed reduces response time; however, due to the overhead created by context switching, the time taken to complete the slow processes increases, thus the system job throughput decreases.

Question 3 - Thread creation, termination, and synchronization

Write a C program to use a group of threads to parallelize the sum of an array.

1. The master threads performs a Serial Sum first. It creates an array of 1,000,000 random floats in $[0, 1]$, and prints out the sum of the array.
2. Then, a group of worker threads are created. Each thread is assigned with a unique ID.
3. Modify the code so that each thread sums up a portion of the array, and a final result is aggregated by the master thread from all threads' results and print out. Briefly describe your design of work distribution among threads.
4. Add 2 timers. One measures the time spent in the serial sum and one measures the parallel sum.
5. Run your program with 4, 8, 12, 16 threads.
6. Make a bar chart to visualize your results.

Solution

To parallelize the summing of the array, we first generate the array and compute the serial sum. The array is filled with random floats in range $[0, 1]$ using `rand()`:

q3.c - Initialization and serial summing of array

```
1 srand(time(NULL));
2 for (int i = 0; i < ARR_SIZE; i++)
3 {
4     arr[i] = (float)rand() / (float)(RAND_MAX);
5 }
6
7 double sum_serial = 0.0;
8 for (int i = 0; i < ARR_SIZE; i++)
9 {
10     sum_serial += arr[i];
11 }
```

For parallelization, each thread computes a partial sum of a slice of the array. All sums are stored in doubles to improve precision. To pass information to the threads we defined a small struct:

q3.c - Struct for thread arguments

```
1 typedef struct{
2     int thread_id;
3     double *partial_sums;
4 } thread_arg_t;
```

- **thread_id**: Thread identifier. Used to compute it's corresponding chunk of the array.
- **partial_sums**: Points to an array allocated in the main thread, where each thread will store it's result in a distinct slot.

Threads are created as follows:

q3.c - Creation of threads

```

1 pthread_t *workers = malloc(sizeof(pthread_t) * num_threads);
2 double *partial_sums = malloc(sizeof(double) * num_threads);
3
4 for (int i = 0; i < num_threads; i++)
5 {
6     thread_arg_t *arg = malloc(sizeof(thread_arg_t));
7     arg->thread_id = i;
8     arg->partial_sums = partial_sums;
9     pthread_create(&workers[i], NULL, thread_func, arg);
10 }
```

Each thread executes a function, **thread_func**, that computes its partial sum and stores it in the allocated array.

q3.c - Thread computation of array partial sum

```

1 void *thread_func(void *arg) {
2     thread_arg_t *data = (thread_arg_t *)arg;
3     int my_id = data->thread_id;
4     double *partial_sums = data->partial_sums;
5     free(arg);
6
7     int chunk_size = ARR_SIZE / num_threads;
8     int start = my_id * chunk_size;
9     int end = (my_id == num_threads - 1) ? ARR_SIZE : start +
    ↪ chunk_size;
10
11     double thread_sum = 0.0;
12     for (int i = start; i < end; i++)
13         thread_sum += arr[i];
14
15     partial_sums[my_id] = thread_sum;
16
17     printf("Thread %d sum = %f\n", my_id, thread_sum);
18     pthread_exit(0);
19 }
```

Once all threads have finished, the main thread sums the partial sums from the threads:

q3.c - Final sum from thread partial sums

```
1 double sum_parallel = 0.0;
2 for (int i = 0; i < num_threads; i++)
3 {
4     sum_parallel += partial_sums[i];
5 }
```

By storing each threads partial sum in the array, we don't need to use mutex locks for access to a global variable. Example output from the application is shown below:

```
> ./q3 8
```

```
1 --- Master Sum ---
2 Serial sum: 500320.812500,    Serial time: 2224 us
3
4 --- Threads ---
5 Thread 0 sum = 62695.894531
6 Thread 1 sum = 62552.230469
7 Thread 2 sum = 62480.328125
8 Thread 3 sum = 62469.359375
9 Thread 4 sum = 62528.675781
10 Thread 5 sum = 62318.031250
11 Thread 6 sum = 62521.257812
12 Thread 7 sum = 62764.535156
13
14 --- Thread Sum ---
15 Parallel Sum: 500330.312500,    Parallel time: 1038 us
```

To measure the performance of the approach and how it scales with the number of threads, a Python script was created that run the application 100 times for `num_threads` $\in [1, 16]$. The result is shown in fig. 6. We see that compared to the simple, serial approach this is some overhead in the creation of threads, as `num_threads = 1` is slightly slower than the baseline. However, as the number of threads is increased we see a performance increase until `num_threads > 6`, whereafter the overhead of thread creation diminishes performance. On our system the optimal number of threads is therefore `num_threads = 6`.

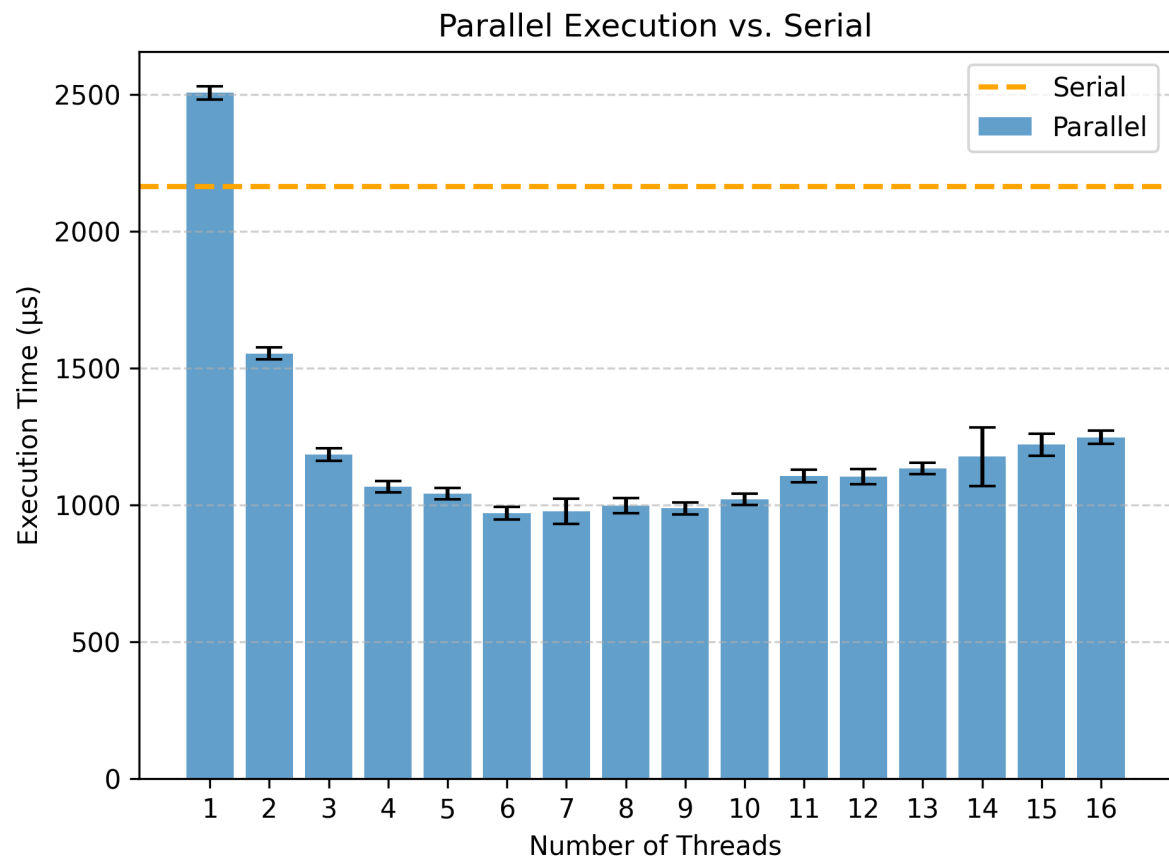


Figure 6: Execution times of array sum program with increased parallelization.

Question 4 - Multi-threaded Histogram

Write a C program to create a group of threads to parallelize building the histogram of an array. You should use 30 bins for the histogram. You can reuse code from Question 3.

1. Modify the code so that it creates an array of doubles, the array length is given by `argv[2]`. The array should be initialized with random values in the range `[0.0, 1.0]`.
2. Serial histogram: add code to create the histogram of the array. Print out the histogram and time.
3. Parallel histogram: Modify the thread function so that each thread builds the histogram of a portion of the array, and a final histogram is aggregated from all threads' result. Print out the histogram and time.
4. Briefly describe your design of shared data structures and synchronization.
5. Run your code: `./program 3 10000`, `./program 3 100000`, `./program 3 1000000`. Include screenshots of your output here.
6. Plots that visualize the output histogram. Analyze your results.

Solution

To parallelize the histogram binning of the array, we first generate the array and compute the serial histogram:

q4.c - Initialization and serial histogram computation

```
1 arr = malloc(sizeof(double) * arr_size);
2 srand(time(NULL));
3 for (int i = 0; i < arr_size; i++)
4 {
5     arr[i] = (double)rand() / (double)(RAND_MAX);
6 }
7
8 int hist_serial[NUM_BINS] = {0};
9 for (int i = 0; i < arr_size; i++)
10 {
11     int bin = (int)(arr[i] * NUM_BINS);
12     if (bin == NUM_BINS) bin = NUM_BINS - 1;
13     hist_serial[bin]++;
14 }
```

For parallelization, the array is split into chunks with each thread computing a partial histogram for its assignment part. To pass thread specific information, a struct is again used:

q4.c - Struct for thread arguments

```

1 typedef struct{
2     int thread_id;
3     int (*partial_histogram)[NUM_BINS];
4 } thread_arg_t;

```

- **thread_id**: Thread identifier. Used to compute it's corresponding chunk of the array.
- **partial_histograms**: Pointer to an array of partial histograms (one per thread).

Threads are created as follows:

q4.c - Creation of threads

```

1 pthread_t *workers = malloc(sizeof(pthread_t) * num_threads);
2 int (*partial_histograms)[NUM_BINS] = calloc(num_threads,
3     ↪ sizeof(*partial_histograms));
4
5 for (int i = 0; i < num_threads; i++)
6 {
7     thread_arg_t *arg = malloc(sizeof(thread_arg_t));
8     arg->thread_id = i;
9     arg->partial_histogram = partial_histograms;
10    pthread_create(&workers[i], NULL, thread_func, arg);
11 }

```

Each thread executes a function, **thread_func**, that computes its partial histogram and stores it in the allocated array:

q4.c - Thread computation of array partial histogram

```

1 void *thread_func(void *arg)
2 {
3     thread_arg_t *data = (thread_arg_t *)arg;
4     int tid = data->thread_id;
5     int (*partial_histograms)[NUM_BINS] = data->partial_histogram;
6     free(arg);
7
8     int chunk_size = arr_size / num_threads;
9     int start = tid * chunk_size;
10    int end = (tid == num_threads - 1) ? arr_size : start + chunk_size;
11
12    for (int i = start; i < end; i++) {
13        int bin = (int)(arr[i] * NUM_BINS);
14        if (bin == NUM_BINS) bin = NUM_BINS - 1;
15        partial_histograms[tid][bin]++;
16    }
17    pthread_exit(NULL);
18 }

```

Once all threads have finished, the main thread adds the partial histograms to form the total histogram:

q4.c - Final histogram from partial histograms

```
1 int hist_parallel[NUM_BINS] = {0};
2 for (int t = 0; t < num_threads; t++)
3 {
4     for (int b = 0; b < NUM_BINS; b++)
5     {
6         hist_parallel[b] += partial_histograms[t][b];
7     }
8 }
```

Just like before, by storing each threads partial histogram in a dedicated array, we don't need mutex locks for access to e.g. a global array. Example output from the application is shown below:

```
> ./q4 3 10000
```

```
1 --- Serial Histogram ---
2 Bin 0: 318   Bin 1: 329   Bin 2: 378   Bin 3: 316   Bin 4: 340
3 Bin 5: 317   Bin 6: 317   Bin 7: 354   Bin 8: 322   Bin 9: 350
4 Bin 10: 363  Bin 11: 329   Bin 12: 314   Bin 13: 315   Bin 14: 276
5 Bin 15: 360  Bin 16: 304   Bin 17: 328   Bin 18: 347   Bin 19: 352
6 Bin 20: 343  Bin 21: 334   Bin 22: 333   Bin 23: 294   Bin 24: 342
7 Bin 25: 363  Bin 26: 338   Bin 27: 366   Bin 28: 343   Bin 29: 315
8 Serial time: 8 us
9
10 --- Parallel Histogram ---
11 Bin 0: 318   Bin 1: 329   Bin 2: 378   Bin 3: 316   Bin 4: 340
12 Bin 5: 317   Bin 6: 317   Bin 7: 354   Bin 8: 322   Bin 9: 350
13 Bin 10: 363  Bin 11: 329   Bin 12: 314   Bin 13: 315   Bin 14: 276
14 Bin 15: 360  Bin 16: 304   Bin 17: 328   Bin 18: 347   Bin 19: 352
15 Bin 20: 343  Bin 21: 334   Bin 22: 333   Bin 23: 294   Bin 24: 342
16 Bin 25: 363  Bin 26: 338   Bin 27: 366   Bin 28: 343   Bin 29: 315
17 Parallel time: 314 us
```

Just like in Question 3, to measure the performance and how it scales with the number of threads, a Python script was used to run each configuration of the application 100 times for $\text{num_threads} \in [1, 16]$. The results are shown in fig. 7. Unfortunately, we don't see a nice improvement from multithreading this application. For all values of N (array size), the serial approach is faster. The cause is most likely that the act of creating the histogram is computationally easy, so the thread overhead always overshadows the gain from parallelization. The high standard deviations however could also point to some randomness from thread creation, or perhaps due to cache misses from bad access to the partial histogram arrays.

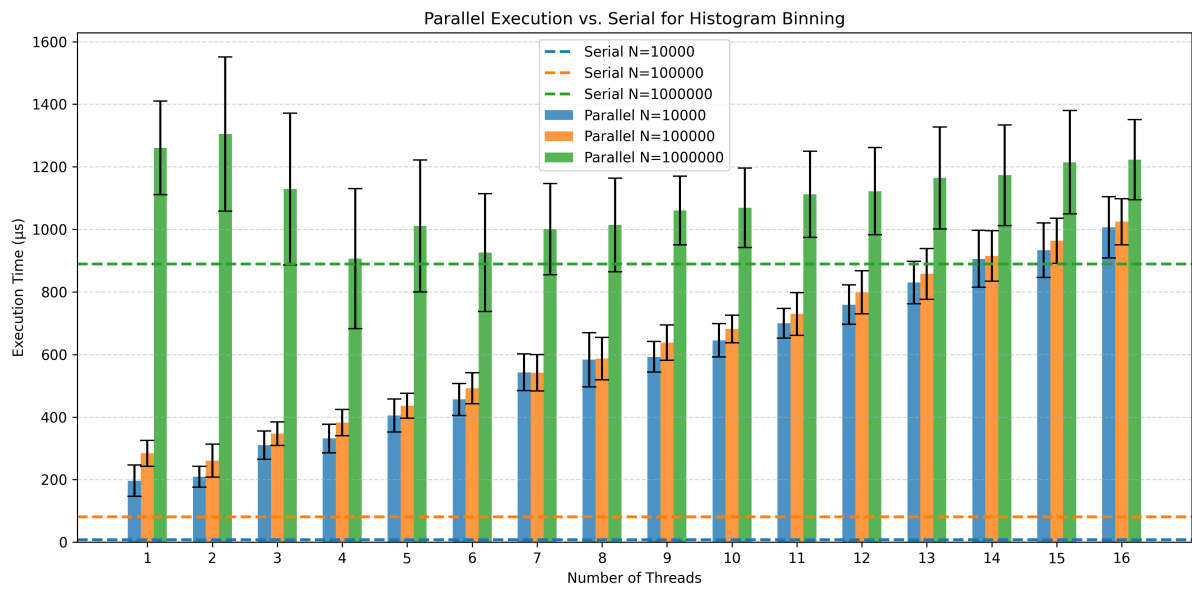


Figure 7: Execution times of array histogram program with increased parallelization.

Question 5 - Lock-free Synchronization

Compare and swap (CAS) instructions leverages hardware atomics on today's computers and can be used to design lock-free data structures. Write a C program to implement Stack as a linked list of Nodes and each thread can perform **push()** and **pop()** on it.

- Each thread performs two functions, **push()** for adding a node to the linked list of nodes, and **pop()** for removing the top node from the linked list. The code should print out the number of nodes remaining in Stack.
- In **pop()** and **push()**, add the implementation based on mutex lock to update top.
- Briefly describe the global data structures in your code and how are access synchronized on them.
- Run your program with 1, 2, 4, 8, 16, 32 threads.
- In **pop()** and **push()**, add the implementation based on compare-and-swap (CAS) operations to update top.
- Briefly describe the global data structures in your code and how are access synchronized on them.
- Run your program with 1, 2, 4, 8, 16, 32 threads.

Solution

We define a simple node struct for the linked-list, along with a global pointer, **top** which always points to the head of the stack. A global counter, **global_node_id** is used to generate unique IDs for each node pushed by the threads.

q5.c - Node structure and global variables

```
1 typedef struct node {
2     int node_id;           // Unique ID for node
3     struct node *next;    // Pointer to next node in stack
4 } Node;
5
6 Node *top = NULL;         // Top of the stack
7 int global_node_id = 0;  // Global counter for node IDs
```

Mutex Approach

In this approach, access to update top of the shared stack is synchronized using a mutex lock. The **push_mutex()** function creates a new node, locks the mutex, and then inserts the node at the top of the stack. Afterwards the lock is released. Similarly we have **pop_mutex()** which removes and frees the top node while ensuring synchronized access by use of a mutex lock.

q5.c - Stack operations using mutex lock

```

1 void push_mutex() {
2     Node *new_node;
3     new_node = malloc(sizeof(Node));
4
5     pthread_mutex_lock(&stack_mutex);
6     new_node->node_id = global_node_id++;
7     new_node->next = top;
8     top = new_node;
9     pthread_mutex_unlock(&stack_mutex);
10 }
11
12 int pop_mutex() {
13     Node *old_node;
14     int node_id = -1;
15
16     pthread_mutex_lock(&stack_mutex);
17     if (top != NULL) {
18         old_node = top;
19         node_id = old_node->node_id;
20         top = top->next;
21         free(old_node);
22     }
23     pthread_mutex_unlock(&stack_mutex);
24     return node_id;
25 }

```

CAS Approach

In this approach we use atomic *Compare-and-Swap* operations. There are two ways we could have approached this in code, either we utilize the `stdatomic.h` library and define atomic variables and utilize their methods, or we directly use the GCC `__sync` intrinsics directly. We chose the latter due to it being slightly clearer what's going on in this particular case. The two intrinsic functions we utilize are [1]:

- `__sync_fetch_and_add(type *ptr, type value)`: Atomically fetches and adds a value to a variable.
- `__sync_bool_compare_and_swap(type *ptr, type oldval, type newval)`: Perform an atomic compare and swap. If the current value of `*ptr` is `oldval`, then write `newval` into `*ptr`. The bool versions returns true if the comparison is successful.

Utilizing the above CAS functions, the top pointer can be updated by the threads without the need of acquiring a lock first. This allows multiple threads to operate concurrently without interfering with each other. In other words, the CAS functions ensure atomicity of the update of `top`. If another thread updates `top` between reading and writing, the CAS fails and the operation is simply retried.

q5.c - Stack operations using compare-and-swap (CAS)

```

1 void push_cas() {
2     Node *new_node;
3     Node *old_top;
4     new_node = malloc(sizeof(Node));
5
6     do {
7         old_top = top;
8         new_node->node_id = __sync_fetch_and_add(&global_node_id, 1);
9         new_node->next = old_top;
10    } while (!__sync_bool_compare_and_swap(&top, old_top, new_node));
11 }
12
13 int pop_cas() {
14     Node *old_top;
15     Node *new_top;
16     int node_id = -1;
17
18     do {
19         old_top = top;
20         if (old_top == NULL)
21         {
22             return -1;
23         }
24         new_top = old_top->next;
25         node_id = old_top->node_id;
26     } while (!__sync_bool_compare_and_swap(&top, old_top, new_top));
27
28     free(old_top);
29     return node_id;
30 }

```

Thread Creation

Like in the previous questions we define a struct for passing arguments to each thread. In this case we pass a Thread ID and an **option** integer that defines whether the thread will operate on the stack using the Mutex or CAS approach:

q5.c Thread argument struct

```

1 typedef struct {
2     int thread_id;
3     int option;    // 0: Mutex, 1: CAS
4 } thread_arg_t;

```

Threads are created in exactly the same way as in the previous questions. Each thread performs the same pattern of stack operations (two pushes, two pops, and one push):

q5.c - Thread function

```

1 void *thread_func(void *arg) {
2     thread_arg_t *data = (thread_arg_t *)arg;
3     int tid = data->thread_id;
4     int opt = data->option;
5
6     if (opt == 0)
7     {
8         push_mutex();
9         push_mutex();
10        pop_mutex();
11        pop_mutex();
12        push_mutex();
13    }
14    else
15    {
16        push_cas();
17        push_cas();
18        pop_cas();
19        pop_cas();
20        push_cas();
21    }
22
23    printf("Thread %d: exit\n", tid);
24    pthread_exit(0);
25 }

```

After all threads are completed, the main thread prints the remaining nodes in the stack:

q5.c - Printing of remaining nodes

```

1 void print_remaining_nodes() {
2     Node *current = top;
3     int count = 0;
4     printf("Remaining nodes: ");
5     while (current != NULL) {
6         printf("%d ", current->node_id);
7         current = current->next;
8         count++;
9     }
10    printf("(Total: %d)\n", count);
11 }

```

The output from running the application with 1, 2, 4, 8, 16, and 32 threads are shown below (for this we disabled the thread exit message). In all cases we see the exact same behavior from the mutex and CAS implementation.

```
> ./q5.c 1
```

```
1 === Stack Using Mutex Approach ===
2 Mutex: Remaining nodes: 2 (Total: 1)
3
4 === Stack Using CAS Approach ===
5 CAS: Remaining nodes: 2 (Total: 1)
```

```
> ./q5.c 2
```

```
1 === Stack Using Mutex Approach ===
2 Mutex: Remaining nodes: 5 2 (Total: 2)
3
4 === Stack Using CAS Approach ===
5 CAS: Remaining nodes: 5 2 (Total: 2)
```

```
> ./q5.c 4
```

```
1 === Stack Using Mutex Approach ===
2 Mutex: Remaining nodes: 11 8 5 2 (Total: 4)
3
4 === Stack Using CAS Approach ===
5 CAS: Remaining nodes: 11 8 5 2 (Total: 4)
```

```
> ./q5.c 8
```

```
1 === Stack Using Mutex Approach ===
2 Mutex: Remaining nodes: 23 20 17 14 11 8 5 2 (Total: 8)
3
4 === Stack Using CAS Approach ===
5 CAS: Remaining nodes: 23 20 17 14 11 8 5 2 (Total: 8)
```

```
> ./q5.c 16
```

```
1 === Stack Using Mutex Approach ===
2 Mutex: Remaining nodes: 47 44 41 38 35 32 29 26 23 20 17 14 11 8 5 2
  ↪ (Total: 16)
3
4 === Stack Using CAS Approach ===
5 CAS: Remaining nodes: 47 44 41 38 35 32 29 26 23 20 17 14 11 8 5 2
  ↪ (Total: 16)
```

```
> ./q5.c 32
```

```
1  === Stack Using Mutex Approach ===
```

```
2  Mutex: Remaining nodes: 95 92 89 86 83 80 77 74 71 68 65 62 59 56 53 50  
   ↪ 47 44 41 38 35 32 29 26 23 20 17 14 11 8 5 2 (Total: 32)
```

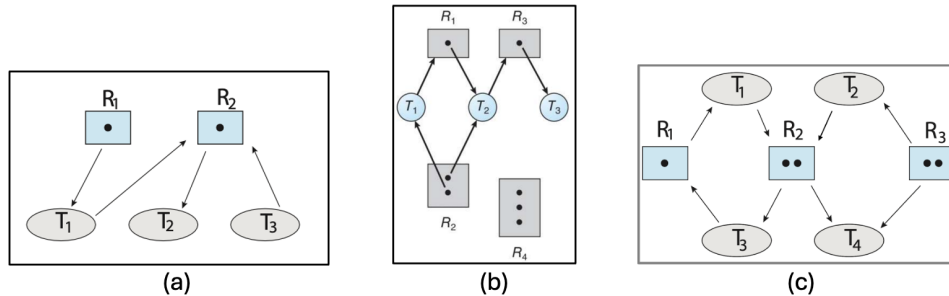
```
3
```

```
4  === Stack Using CAS Approach ===
```

```
5  CAS: Remaining nodes: 95 92 89 86 83 80 77 74 71 68 65 62 59 56 53 50  
   ↪ 47 44 41 38 35 32 29 26 23 20 17 14 11 8 5 2 (Total: 32)
```

Question 6 - Resource-Allocation Graph

Recall that a resource-allocation graph without cycles has no deadlock, and a resource-allocation graph with cycles may or may not have deadlocks. In the following resource-allocation graphs:



- Please describe thread states in each graph. For example, thread T_0 is requesting an instance of resource R_0 , thread T_1 is holding an instance of resource R_1 .
- Which graph(s) have no deadlocks? Please illustrate the order in which the threads may complete execution. For example, thread T_0 releases an instance of resource R_0 , and then an instance of resource R_0 is allocated to thread T_1 .
- Which graph may have deadlocks? Recall that a cycle in the resource-allocation graph does not necessarily mean deadlock. For those situations that are deadlocked, provide the cycle of threads and resources. For example, $\{T_0 \rightarrow R_0 \rightarrow T_1 \rightarrow R_1 \rightarrow T_0\}$.

Solution

a)

In Figure A, T_1 has been allocated 1 instance of R_1 and requests 1 instance of R_2 , T_2 has been allocated 1 instance of R_2 , and T_3 requests 1 instance of R_2 . In Figure B, T_1 requests R_1 and has been allocated 1 instance of R_2 , T_2 requests 1 instance of R_3 and has been allocated 1 instance of R_2 and R_1 , and T_3 has been allocated 1 instance of R_3 . In Figure C, T_1 requests 1 instance of R_2 and has been allocated 1 instance of R_1 , T_2 requests 1 instance of R_2 and has been allocated 1 instance of R_3 , T_3 requests 1 instance of R_1 and has been allocated 1 instance of R_2 , T_4 has been allocated with both 1 instance of R_2 and 1 instance of R_3 .

b)

Figures A and B do not have deadlocks, as they do not contain any cycles. Figure C, although having cycles, does not have deadlocks.

For Figure A, the order can be: T_2 releases 1 instance of R_2 ; 1 instance of R_2 is allocated to T_3 ; T_3 releases 1 instance of R_2 ; 1 instance of R_2 is allocated to T_1 ; T_1 releases 1 instance of R_2 and R_1 .

For Figure B, the order can be: T_3 releases 1 instance of R_3 ; 1 instance of R_3 is allocated to T_2 ; T_2 releases 1 instance of R_1 and R_2 ; 1 instance of R_1 is allocated to T_1 ; T_1 releases 1 instance of R_1 and R_2 .

For Figure C, the order can be: T4 releases 1 instance of R2 and R3; 1 instance of R2 is allocated to T2; T2 releases 1 instance of R2 and R3; 1 instance of R2 is allocated to T1; T1 releases 1 instance of R1 and R2; 1 instance of R2 is allocated to T3; T3 releases 1 instance of R1 and R2.

c)

Figure C may have deadlocks, as it is the only one with cycles in its graph. However, T4 should eventually release R3 and R2, causing R1 accessible to T1. Once T1 is completed and releases R1 and R2, no cycle can be found again. Thus, there is no actual deadlock in this figure.

Question 7 - Banker's Algorithm

Given the following system snapshot:

Processes	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2	1	5	2	0
P1	1	0	0	0	1	7	5	0				
P2	0	3	5	4	2	3	5	6				
P3	0	6	3	2	0	6	5	2				
P4	0	0	1	4	0	6	5	6				

Table 1: System Snapshot

- List the number of instances per resource. List down the need matrix.
- Assume process P1 requests for (0, 4, 2, 0), can the request be granted immediately? Explain your answer.
- Run the Banker's safety check algorithm. Answer whether the system is in a safe state. Explain your answer.
- Identify a safe sequence of processes.

Solution

a)

The total number of instances per resource is calculated as the sum of allocated resources plus the currently available resources:

$$R0 = (0 + 1 + 0 + 0 + 0) + 1 = 2$$

$$R1 = (0 + 0 + 3 + 6 + 0) + 5 = 14$$

$$R2 = (1 + 0 + 5 + 3 + 1) + 2 = 12$$

$$R3 = (2 + 0 + 4 + 2 + 4) + 0 = 12$$

Therefore, the total number of instances for each resource type is:

$$R0 = 2, \quad R1 = 14, \quad R2 = 12, \quad R3 = 12$$

The Need Matrix represents the remaining resources each process still needs to complete its execution. It is calculated as Need = Max - Allocation:

Process	R0	R1	R2	R3
P0	$0 - 0 = 0$	$0 - 0 = 0$	$1 - 1 = 0$	$2 - 2 = 0$
P1	$1 - 1 = 0$	$7 - 0 = 7$	$5 - 0 = 5$	$0 - 0 = 0$
P2	$2 - 1 = 1$	$3 - 3 = 0$	$5 - 5 = 0$	$6 - 4 = 2$
P3	$0 - 0 = 0$	$6 - 6 = 0$	$5 - 3 = 2$	$2 - 2 = 0$
P4	$0 - 0 = 0$	$6 - 0 = 6$	$5 - 1 = 4$	$6 - 4 = 2$

Table 2: Need Matrix

b)

Let

$$\text{Request}_{P1} = (0, 4, 2, 0)$$

Step 1: Check if Request $\leq$ Need

The request must not exceed P1's maximum declared need:

$$\text{Need}_{P1} = (0, 7, 5, 0), \quad \text{Request}_{P1} = (0, 4, 2, 0)$$

$$0 \leq 0, \quad 4 \leq 7, \quad 2 \leq 5, \quad 0 \leq 0$$

All conditions are satisfied. Therefore,

$$\text{Request}_{P1} \leq \text{Need}_{P1}$$

Step 2: Check if Request $\leq$ Available

The request must not exceed the currently available resources:

$$\text{Available} = (1, 5, 2, 0), \quad \text{Request}_{P1} = (0, 4, 2, 0)$$

$$0 \leq 1, \quad 4 \leq 5, \quad 2 \leq 2, \quad 0 \leq 0$$

All conditions are satisfied. Therefore,

$$\text{Request}_{P1} \leq \text{Available}$$

Since both conditions are met, the request for P1 can be granted immediately in terms of current availability and P1's declared maximum need. However, to ensure the system remains in a safe state after granting the request, we must perform a safety check.

c)

Let's assume P1's request for (0, 4, 2, 0) is granted. The system state would change as follows:

New Allocation for P1:

$$\text{Old Allocation}_{P1} = (1, 0, 0, 0)$$

$$\text{Request}_{P1} = (0, 4, 2, 0)$$

$$\text{New Allocation}_{P1} = (1 + 0, 0 + 4, 0 + 2, 0 + 0) = (1, 4, 2, 0)$$

New Available Resources:

$$\text{Old Available} = (1, 5, 2, 0)$$

$$\text{Request}_{P1} = (0, 4, 2, 0)$$

$$\text{New Available} = (1 - 0, 5 - 4, 2 - 2, 0 - 0) = (1, 1, 0, 0)$$

New Need for P1:

$$\text{Old Need}_{P1} = (0, 7, 5, 0)$$

$$\text{Request}_{P1} = (0, 4, 2, 0)$$

$$\text{New Need}_{P1} = (0 - 0, 7 - 4, 5 - 2, 0 - 0) = (0, 3, 3, 0)$$

Now, we run the Banker's Safety Algorithm with the new system state:

Updated Allocation Matrix:

Process	R0	R1	R2	R3
P0	0	0	1	2
P1	1	4	2	0
P2	1	3	5	4
P3	0	6	3	2
P4	0	0	1	4

Table 3: Updated Allocation Matrix

Updated Need Matrix:

Process	R0	R1	R2	R3
P0	0	0	0	0
P1	0	3	3	0
P2	1	0	0	2
P3	0	0	2	0
P4	0	6	4	2

Table 4: Updated Need Matrix

Updated Available Vector:

$$\text{Available}_{new} = (1, 1, 0, 0)$$

Safety Algorithm Steps:

1. Initialize $\text{Work} = \text{Available}_{new} = (1, 1, 0, 0)$
2. Initialize $\text{Finish} = [\text{False}, \text{False}, \text{False}, \text{False}, \text{False}]$ for P0 to P4.

Iteration 1

- P0: $\text{Need} = (0, 0, 0, 0)$. Is $\text{Need}_{P0} \leq \text{Work}$? $(0 \leq 1, 0 \leq 1, 0 \leq 0, 0 \leq 0) \rightarrow \text{True}$.
 - $\text{Work} = \text{Work} + \text{Allocation}_{P0} = (1, 1, 0, 0) + (0, 0, 1, 2) = (1, 1, 1, 2)$
 - $\text{Finish}[P0] = \text{True}$

Iteration 2

- P1: Need = (0, 3, 3, 0). Is $\text{Need}_{P1} \leq \text{Work}$? $(0 \leq 1, 3 \leq 1, 3 \leq 1, 0 \leq 2) \rightarrow \text{False}$ ($3 > 1$)
- P2: Need = (1, 0, 0, 2). Is $\text{Need}_{P2} \leq \text{Work}$? $(1 \leq 1, 0 \leq 1, 0 \leq 1, 2 \leq 2) \rightarrow \text{True}$.
 - $\text{Work} = \text{Work} + \text{Allocation}_{P2} = (1, 1, 1, 2) + (1, 3, 5, 4) = (2, 4, 6, 6)$
 - $\text{Finish}[P2] = \text{True}$
- P1: Need = (0, 3, 3, 0). Is $\text{Need}_{P1} \leq \text{Work}$? $(0 \leq 2, 3 \leq 4, 3 \leq 6, 0 \leq 6) \rightarrow \text{True}$.
 - $\text{Work} = \text{Work} + \text{Allocation}_{P1} = (2, 4, 6, 6) + (1, 4, 2, 0) = (3, 8, 8, 6)$
 - $\text{Finish}[P1] = \text{True}$
- P3: Need = (0, 0, 2, 0). Is $\text{Need}_{P3} \leq \text{Work}$? $(0 \leq 3, 0 \leq 8, 2 \leq 8, 0 \leq 6) \rightarrow \text{True}$.
 - $\text{Work} = \text{Work} + \text{Allocation}_{P3} = (3, 8, 8, 6) + (0, 6, 3, 2) = (3, 14, 11, 8)$
 - $\text{Finish}[P3] = \text{True}$
- P4: Need = (0, 6, 4, 2). Is $\text{Need}_{P4} \leq \text{Work}$? $(0 \leq 3, 6 \leq 14, 4 \leq 11, 2 \leq 8) \rightarrow \text{True}$.
 - $\text{Work} = \text{Work} + \text{Allocation}_{P4} = (3, 14, 11, 8) + (0, 0, 1, 4) = (3, 14, 12, 12)$
 - $\text{Finish}[P4] = \text{True}$

All processes have $\text{Finish}[i] = \text{True}$.

Conclusion: Yes, the system is in a safe state after P1's request for (0, 4, 2, 0) is granted, because a safe sequence exists.

d)

Based on the safety check above, a safe sequence is:

$$\langle P0, P2, P1, P3, P4 \rangle$$

This sequence demonstrates that all processes can complete their execution without causing a deadlock, even after P1's request is granted.

Question 8 - Minimum Resources

Given the following system snapshot:

Processes	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	1	0	1	2	1	0	1	2	0	x	4	1
P1	1	0	3	0	1	7	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	2	3	2	0	6	5	2				

Table 5: System Snapshot

Determine the smallest value of x for which this is a safe state. Briefly explain how you derive the answer.

Solution

Step 1: Compute the Need Matrix

The Need matrix is calculated as $\text{Need} = \text{Max} - \text{Allocation}$.

$$\text{P0 Need} = [1, 0, 1, 2] - [1, 0, 1, 2] = [0, 0, 0, 0]$$

$$\text{P1 Need} = [1, 7, 5, 0] - [1, 0, 3, 0] = [0, 7, 2, 0]$$

$$\text{P2 Need} = [2, 3, 5, 6] - [1, 3, 5, 4] = [1, 0, 0, 2]$$

$$\text{P3 Need} = [0, 6, 5, 2] - [0, 2, 3, 2] = [0, 4, 2, 0]$$

Process	Need
P0	[0, 0, 0, 0]
P1	[0, 7, 2, 0]
P2	[1, 0, 0, 2]
P3	[0, 4, 2, 0]

Table 6: Need Matrix

Step 2: Determine a Safe Sequence Using the Banker's Algorithm

For a state to be safe, there must exist a sequence of processes such that each process's Need is less than or equal to the Available resources. When a process executes, its Allocation is added back to Available.

Let the initial Available vector be $[0, x, 4, 1]$. We try to find a safe sequence:

Trial 1: P0

$$\text{Need}_{P0} = [0, 0, 0, 0], \quad \text{Available} = [0, x, 4, 1]$$

Since $\text{Need}_{P0} \leq \text{Available}$ for any $x \geq 0$, P0 can execute.

$$\text{Available} = \text{Available} + \text{Allocation}_{P0} = [0, x, 4, 1] + [1, 0, 1, 2] = [1, x, 5, 3]$$

Trial 2: P3

$$\text{Need}_{P3} = [0, 4, 2, 0], \quad \text{Available} = [1, x, 5, 3]$$

For P3 to execute, $[0, 4, 2, 0] \leq [1, x, 5, 3]$, which requires $x \geq 4$. If P3 executes:

$$\text{Available} = [1, x, 5, 3] + [0, 2, 3, 2] = [1, x + 2, 8, 5]$$

Trial 3: P2

$$\text{Need}_{P2} = [1, 0, 0, 2], \quad \text{Available} = [1, x + 2, 8, 5]$$

Since $[1, 0, 0, 2] \leq [1, x + 2, 8, 5]$, P2 can execute.

$$\text{Available} = [1, x + 2, 8, 5] + [1, 3, 5, 4] = [2, x + 5, 13, 9]$$

Trial 4: P1

$$\text{Need}_{P1} = [0, 7, 2, 0], \quad \text{Available} = [2, x + 5, 13, 9]$$

To execute, $[0, 7, 2, 0] \leq [2, x + 5, 13, 9]$ requires $7 \leq x + 5 \Rightarrow x \geq 2$.

Combining Conditions:

From P3: $x \geq 4$

From P1: $x \geq 2$

Therefore, the smallest value of x that satisfies all conditions is $x = 4$.

Safe Sequence:

A possible safe sequence for $x = 4$ is:

$$\langle P0, P3, P2, P1 \rangle$$

References

- [1] I. Free Software Foundation, “*__sync builtins — using the gnu compiler collection (gcc)*”, https://gcc.gnu.org/onlinedocs/gcc-4.7.1/gcc/_005f_005fsync-Builtins.html, Accessed: 2025-10-14.